

Immediate Database Modification

Log-Based Recovery, UNDO/REDO Mechanisms, and Comparative Analysis

1. WHAT IS IMMEDIATE DATABASE MODIFICATION?

In Immediate Database Modification, uncommitted transaction updates are written directly to the active database in secondary storage **before** the transaction executes its `COMMIT` operation.

★ Golden Rule

Immediate Modification = Update Database Immediately (without waiting for `COMMIT`). This stands in direct contrast to Deferred Database Modification, which delays disk writes until commit completion.

2. TRANSACTION EXECUTION EXAMPLE

Consider initial database values: `A = 100`, `B = 200`. Transaction T_1 executes:

STEP 1 Update A

```
READ(A)
A = A + 100
WRITE(A)
Result: A = 200 is written immediately to disk (Pre-Commit).
```

STEP 2 Update B

```
READ(B)
B = B + 200
WRITE(B)
Result: B = 400 is written immediately to disk (Pre-Commit).
```

3. TRANSACTION LOG REQUIREMENTS

Because database records are modified on disk prior to commit, the log must store **both Old Values (V_{old}) and New Values (V_{new})** to support recovery operations.

```
<T1 START>
<T1, A, 100, 200> <-- <Txn_ID, Data_Item, Old_Value, New_Value>
<T1, B, 200, 400>
<T1 COMMIT>
```

4. RECOVERY DYNAMICS UNDER FAILURE

Case A: Failure AFTER Commit

Log shows both `<T1 START>` and `<T1 COMMIT>`.

Action: REDO T1

Re-applies new values (`A = 200`, `B = 400`) to ensure persistence.

Case B: Failure BEFORE Commit

Log shows `<T1 START>` but **NO COMMIT**.

Action: UNDO T1

Restores old values (`A = 100`, `B = 200`) to wipe out uncommitted changes.

5. CORE INVARIANT

Why are old values strictly required? Because uncommitted modifications have already overwritten original disk data. If a transaction aborts, the original values can only be recovered from the log.

Immediate Database Modification = UNDO + REDO

6. MULTI-TRANSACTION FAILURE EXAMPLE

Consider the following transaction log sequence at the time of a crash:

```
<T1 START>
<T1, A, 100, 200>
<T1 COMMIT>
<T2 START>
<T2, B, 200, 400>
💣 SYSTEM FAILURE
```

RECOVERY ANALYSIS

- Transaction T_1 : Has both **START** and **COMMIT** in log.
→ Action: **REDO**(T_1)
- Transaction T_2 : Has **START** but **NO COMMIT** in log.
→ Action: **UNDO**(T_2)

7. RECOVERY DECISION RULES

TRANSACTION LOG STATE	RECOVERY ACTION TAKEN	REASONING
<Ti START> AND <Ti COMMIT>	🔄 REDO	Transaction completed; ensure new values are written.
<Ti START> AND NO COMMIT	↩️ UNDO	Transaction uncommitted; restore original pre-crash values.

8. IMMEDIATE VS. DEFERRED MODIFICATION

FEATURE	IMMEDIATE MODIFICATION	DEFERRED MODIFICATION
DB Updated Before Commit?	✅ Yes	❌ No
Log Stores Old Values?	✅ Yes (Old + New)	❌ No (New only)
UNDO Operation Required?	✅ Yes	❌ No
REDO Operation Required?	✅ Yes	✅ Yes
Recovery Strategy	UNDO + REDO	REDO only

9. EXAM TRAPS & KEY CONCEPTS ⚠️

Trap 1: "Immediate" Meaning

Immediate modification does **NOT** mean the transaction commits immediately. It means disk writes occur immediately during execution.

Trap 2: Why Old Values?

Old values are required solely for the **UNDO** operation when an uncommitted transaction crashes after writing to disk.

10. MEMORY CHEATSHEET & 10-SECOND REVISION

⚡ 10-SECOND REVISION FLOW

Write Changes to DB Immediately (Before COMMIT)



Log Stores [OLD + NEW] Values



On System Failure:

Committed → **REDO** | Uncommitted → **UNDO**

🔑 **Immediate = Before Commit + Old/New Values + UNDO/REDO**